

# Frontend Component Architecture — Family Music Scrobbler PWA

## Frontend Component Architecture - Family Music Scrobbler PWA

A complete architectural reference for the React-based Progressive Web App (PWA) powering the Family Music Scrobbler. This document defines the component hierarchy, state management strategy, offline behaviour, and integration points with the backend API.

---

### 1. Overview

The frontend is a **React PWA** designed for:

- Fast, responsive UI across desktop and mobile
- Offline-first behaviour using Service Worker + IndexedDB
- Background sync for queued updates
- Push notifications for leaderboard updates
- Modular, maintainable component structure

The architecture follows a **feature-first** directory layout.

---

## 2. Directory Structure

```
frontend/src/  
  components/  
    tracks/  
    leaderboard/  
    auth/  
    layout/  
  pages/  
    HomePage.jsx  
    RecentTracksPage.jsx  
    LeaderboardPage.jsx  
    ProfilePage.jsx  
    SettingsPage.jsx  
  hooks/  
    useAuth.js  
    useApi.js  
    useIndexedDB.js  
    useServiceWorker.js  
  lib/  
    api.js  
    storage.js  
    notifications.js  
  service-worker.js  
  manifest.json
```

---

## 3. Component Architecture

### 3.1 Layout Components

Reusable UI scaffolding.

#### Layout/Header

- Navigation
- User avatar
- Online/offline indicator

#### Layout/Footer

- Version info
- PWA install prompt

## Layout/Container

- Standardised page wrapper
- 

## 3.2 Authentication Components

### Auth/LoginForm

- Local login
- Redirect to Spotify OAuth

### Auth/RegisterForm

- Local registration

### Auth/LogoutButton

- Clears tokens
  - Flushes IndexedDB
- 

## 3.3 Track Components

### Tracks/RecentTrackList

Displays recent tracks.

Props:

```
tracks: Array<Track>
loading: boolean
```

### Tracks/TrackCard

Single track display.

Props:

```
track: Track
```

### Tracks/TrackSkeleton

Loading placeholder.

---

## 3.4 Leaderboard Components

### Leaderboard/WeeklyLeaderboard

Displays weekly play counts.

Props:

```
entries: Array<LeaderboardEntry>
```

### **Leaderboard/UserRow**

Single leaderboard row.

Props:

```
user: User  
playCount: number
```

---

## **3.5 Profile & Settings Components**

### **Profile/ProfileCard**

Displays user profile.

### **Profile/EditProfileForm**

Allows editing display name + leaderboard opt-out.

### **Settings/SettingsPanel**

- PWA install button
  - Notification toggle
  - Offline data inspector
- 



## **4. Pages**

### **HomePage**

- Overview
- Quick links
- Online/offline status

### **RecentTracksPage**

- Fetches /tracks/recent
- Displays RecentTrackList
- Uses network-first caching

### **LeaderboardPage**

- Fetches /analytics/weekly-leaderboard
- Displays WeeklyLeaderboard

## ProfilePage

- Fetches /auth/me
- Displays ProfileCard

## SettingsPage

- Manages PWA + notifications
- Displays offline sync queue

---

## 5. State Management Strategy

### Global State

Managed via React Context:

- Auth state
- Online/offline state
- Push notification subscription

### Local State

Managed via component-level hooks.

### Persistent State

Stored in IndexedDB:

- Offline sync queue
- Cached recent tracks
- Cached leaderboard entries

---

## 6. API Integration

All API calls go through lib/api.js.

### Example

```
export async function getRecentTracks() {  
  return api.get('/tracks/recent');  
}
```

### Features

- Automatic token injection

- Retry logic
  - Network-first strategy
  - Background sync fallback
- 

## 7. Offline Architecture

### **Service Worker**

Handles:

- Cache-first for app shell
- Network-first for API
- Push notifications

### **IndexedDB**

Stores:

- Cached API responses
- Sync queue

### **Background Sync API**

Automatically retries queued requests.

---

## 8. Notifications Architecture

### **Push API**

Used for:

- Leaderboard updates
- Ingestion alerts

### **VAPID Keys**

Stored in Docker secrets.

### **Notification Flow**

1. Backend sends push event
  2. Service Worker receives event
  3. Notification displayed
  4. UI updates leaderboard
-

## 9. Testing Strategy

### **Unit Tests**

- Component rendering
- Hooks behaviour
- API wrapper

### **Integration Tests**

- Page-level flows
- Offline behaviour
- IndexedDB operations

### **PWA Tests**

- Service Worker registration
- Cache behaviour
- Background sync



## 10. Summary

This architecture ensures:

- Modular, maintainable React components
- Reliable offline-first behaviour
- Smooth integration with backend APIs
- Scalable UI for long-term analytics

It provides a solid foundation for future enhancements such as:

- Advanced charts
- Multi-account dashboards
- Cross-service ingestion (YouTube Music)